



Nota: Este teste tem 2 partes que devem ser respondidas em 2 cadernos distintos para permitir que os docentes da UCs possam corrigir em paralelo cada uma das 2 partes.

Parte I (35% - 70 pontos)

- ²⁰O paralelismo pode ser explorado a vários níveis: instruções, fios de execução (i.e., *threads*) e processos. **Caracterize** sumariamente cada um destes níveis de paralelismo, salientando as diferenças mais relevantes entre os vários níveis.
- ¹⁰**Explique** o que é e para que serve o modelo “roofline”.
- ¹⁰**Indique**, justificando, quantos “outputs” diferentes se pode obter com o seguinte programa OpenMP (considere que cada “printf” origina uma linha):

```
int main() {  
    printf("master thread\n");  
    #pragma omp parallel num_threads(2)  
    {  
        int id = omp_get_thread_num();  
        #pragma omp for schedule(static)  
        for(int i=0; i<6; i++) {  
            printf("T%d:i%d\n", id, i);  
            #pragma omp barrier  
        }  
    }  
    printf("master thread\n");  
}
```

- ²⁰As GPUs ganharam nos últimos anos uma grande popularidade. Apesar de assentarem num modelo de memória partilhada apresentam diferenças significativas relativamente às arquiteturas tradicionais (e.g., “multicore”). **Discuta** essas diferenças com base nos seguintes pontos:
 - ¹⁰Número de fios de execução a especificar e a forma como são escalonados.
 - ¹⁰Gestão da localidade dos dados.
- ¹⁰O suporte a novos formatos de vírgula flutuante, designadamente o FP16 e o FP8 permitem aumentar consideravelmente o desempenho dos sistemas de computação. **Explique** como é que estes novos formatos permitem melhorar o desempenho e quais as principais limitações à sua utilização na generalidade das aplicações.

Parte II (65% -130 pontos)

Considere o programa seguinte, com várias funções, que aplica um conjunto de transformações a uma imagem, representada por uma matriz bidimensional de valores inteiros entre 0 e 255.

```
#define N 10000

int* sum_columns (int* image) {
    int *result = init_and_zero(sizeof(int)*N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            result[j] += image[i*N+j] ;
        }
    }
    return result;
}

int* sum_lines (int* image) {
    int *result = init_and_zero(sizeof(int)*N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if(image[i*N+j]<100)
                result[i] += image[i*N+j] ;
        }
    }
    return result;
}

int reduceAll (int* image) {
    int result = 0;
    for (int i = 0; i < N*N; i++) {
        result += image[i];
    }
    return result;
}

void multiplication (int* image, int *lines, int *columns, int all) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            image[i*N+j] = (columns[i]*lines[j]) / all;
        }
    }
}

int main() {
    // ... read image ...

    int *lines, *columns, all;
    all = reduceAll(image);
    lines = sum_lines(image);
    columns = sum_columns(image);
    multiplication(image, lines, columns, all);

    // ... save image ...
}
```

1. ³⁰Pretende-se analisar e otimizar a versão sequencial deste programa.
 - a) ¹⁵**Indique** a complexidade de cada uma das funções invocadas na *main*.
 - b) ¹⁵**Apresente** duas otimizações possíveis e **discuta** o seu impacto no tempo de execução.

2. ⁴⁰Pretende-se desenvolver uma versão paralela com **OpenMP**.
 - a) ¹⁵**Indique**, justificando, qual das funções `sum_columns` ou `reduceAll` retirará melhor proveito do paralelismo.
 - b) ²⁵**Desenvolva** uma versão paralela eficiente da função `sum_lines` e **comente** o propósito de cada uma das diretivas que usar neste contexto.

3. ³⁰Pretende-se desenvolver um *kernel* em CUDA para implementar a função `multiplication`. Considere o *kernel* CUDA apresentado em baixo e ignore a chamada do *kernel*, e as cópias de memória entre *host* e *device*.

```
__global__
void multiplication(int *image, int *lines, int *columns, int all) {
    int thread_id_x = threadIdx.x + blockIdx.x * blockDim.x;
    ...
}
```

 - a) **Complete** o *kernel* CUDA apresentado. Quantas *threads* deverão ser lançadas para executar este *kernel*? Justifique.

4. ³⁰Pretende-se desenvolver uma versão paralela do código para memória distribuída com **passagem de mensagens (i.e., MPI)** e em que o processo 0 contém a imagem a processar e, no final, deverá conter a imagem processada (`image`).
 - a) ¹⁵**Identifique**, justificando, qual o padrão que se adequa melhor à resolução do problema com MPI, a partir dos padrões paralelos *pipeline* e *farm*. **Apresente** um diagrama com as trocas de mensagens necessárias entre os vários processos, para o padrão escolhido (**ilustre** com 3 processos).
 - b) ¹⁵**Indique**, justificando, as diretivas de comunicação a usar para implementar o padrão de comunicação proposto na alínea anterior.